

Name: Gaurika Mittal

Batch ID: 15

Sap ID: 590034892

Experiment-13

Question

Write a **menu-driven C program using switch-case** to input a positive integer and perform one of the following operations based on the user's choice:

1. **Check whether all its digits are unique.**
 - Example 1: Number = 12345
Output: Unique
 - Example 2: Number = 12234
Output: Not Unique
2. **Find the frequency of each digit from 0 to 9.**
 - Example: Number = 112333
Output:
freq_0 = 0
freq_1 = 2
freq_2 = 1
freq_3 = 3
freq_4 = 0
freq_5 = 0
freq_6 = 0
freq_7 = 0
freq_8 = 0
freq_9 = 0

1. Algorithm

Algorithm for Menu-Driven Program

1. Start.
2. Declare variables `num`, `choice`, `temp`, `digit`, `i` and an array `freq[10]`.
3. Input a positive integer from the user.
4. Display the menu:
 - 1. Check whether all digits are unique.
 -

2. Find frequency of each digit.
5. Input the user's choice.
6. Use `switch(choice)` to perform the selected operation.
7. If **choice = 1**:
 - Initialize all elements of `freq[10]` to 0.
 - Copy `num` into `temp`.
 - Extract each digit using `temp % 10`.
 - Increase the frequency of that digit.
 - If any digit occurs more than once, display "**Not Unique**".
 - Otherwise, display "**Unique**".
8. If **choice = 2**:
 - Initialize all elements of `freq[10]` to 0.
 - Copy `num` into `temp`.
 - Extract each digit using `temp % 10`.
 - Increase the corresponding frequency.
 - Display the frequency of digits from 0 to 9.
9. If the choice is neither 1 nor 2, display "**Invalid choice**".
10. Stop.

2. Test Case Table

Test Case	Inputs	Expected Outputs	Actual Outputs	Results
1	Number = 12345, Choice = 1	Unique	Unique	Pass
2	Number = 12234, Choice = 1	Not Unique	Not Unique	Pass
3	Number = 98760, Choice = 1	Unique	Unique	Pass
4	Number = 112333, Choice = 2	freq_0=0, freq_1=2, freq_2=1, freq_3=3, others=0	freq_0=0, freq_1=2, freq_2=1, freq_3=3, others=0	Pass
5	Number = 50505, Choice = 2	freq_0=2, freq_5=3, others=0	freq_0=2, freq_5=3, others=0	Pass

3. C Program

```
#include <stdio.h>
int main()
{
    int num, choice, temp, digit, i;
    int freq[10] = {0};
```

```

int unique = 1;
printf("Enter a positive integer: ");
scanf("%d", &num);
printf("\n1. Check whether all digits are unique");
printf("\n2. Find frequency of each digit");
printf("\nEnter your choice: ");
scanf("%d", &choice);

switch(choice)
{
    case 1:
        temp = num;
        while(temp > 0)
        {
            digit = temp % 10;
            freq[digit]++;
            if(freq[digit] > 1)
            {
                unique = 0;
                break;
            }
            temp = temp / 10;
        }
        if(unique == 1)
            printf("Unique\n");
        else
            printf("Not Unique\n");
        break;
    case 2:
        temp = num;
        while(temp > 0)
        {
            digit = temp % 10;
            freq[digit]++;
            temp = temp / 10;
        }
        for(i = 0; i < 10; i++)
        {
            printf("freq_%d = %d\n", i, freq[i]);
        }
        break;
    default:
        printf("Invalid choice\n");
}
return 0;
}

```

4. Compilation and Execution

```
(kali㉿kali)-[~]  
$ gcc hdoc14.c -o hdoc14  
  
(kali㉿kali)-[~]  
$ ./hdoc14  
Enter a positive integer: 12345  
  
1. Check whether all digits are unique  
2. Find frequency of each digit  
Enter your choice: 1  
Unique
```

```
(kali㉿kali)-[~]  
$ gcc hdoc14.c -o hdoc14  
  
(kali㉿kali)-[~]  
$ ./hdoc14  
Enter a positive integer: 12234  
  
1. Check whether all digits are unique  
2. Find frequency of each digit  
Enter your choice: 1  
Not Unique
```

```
(kali㉿kali)-[~]  
$ gcc hdoc14.c -o hdoc14  
  
(kali㉿kali)-[~]  
$ ./hdoc14  
Enter a positive integer: 98760  
  
1. Check whether all digits are unique  
2. Find frequency of each digit  
Enter your choice: 1  
Unique
```

```
(kali㉿kali)-[~]  
$ gcc hdoc14.c -o hdoc14  
  
(kali㉿kali)-[~]  
$ ./hdoc14  
Enter a positive integer: 112333  
  
1. Check whether all digits are unique  
2. Find frequency of each digit  
Enter your choice: 2  
freq_0 = 0  
freq_1 = 2  
freq_2 = 1  
freq_3 = 3  
freq_4 = 0  
freq_5 = 0  
freq_6 = 0
```

```
freq_7 = 0  
freq_8 = 0  
freq_9 = 0
```

```
(kali㉿kali)-[~]  
$ gcc hdoc14.c -o hdoc14
```

```
(kali㉿kali)-[~]  
$ ./hdoc14
```

Enter a positive integer: 50505

1. Check whether all digits are unique
2. Find frequency of each digit

Enter your choice: 2

```
freq_0 = 2  
freq_1 = 0  
freq_2 = 0  
freq_3 = 0  
freq_4 = 0  
freq_5 = 3  
freq_6 = 0  
freq_7 = 0  
freq_8 = 0  
freq_9 = 0
```